

C Program

1. Program to sort the given list using selection sort technique.

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i,j,n,a[20],small,pos;
    clrscr();
    printf("Enter the number of elements\n");
    scanf("%d",&n);
    printf("Enter the array element\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    for(i=0;i<n;i++)
    {
        small=a[i];
        pos=i;
        for(j=i+1;j<n;j++)
        {
            if(a[j]<small)
            {
                small=a[j];
                pos=j;
            }
        }
        a[pos]=a[i];
        a[i]=small;
    }
    printf("\n Sorted array is ");
    for(i=0;i<n;i++)
    {
        printf("%d\t",a[i]);
    }
    getch();
}
```

-
2. Program to sort the given list using insertion sort technique

```

#include<stdio.h>
#include<conio.h>
void main()
{
    int a[20],j,temp,i,n;
    clrscr();
    printf("Enter the size of the array\n");
    scanf("%d",&n);
    printf("Enter the elements into the array\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    for(i=0;i<n;i++)
    {
        temp=a[i];
        for(j=i-1;j>=0;j--)
        {
            if(a[j]>temp)
            {
                a[j+1]=a[j];
            }
            else
            {
                break;
            }
        }
        a[j+1]=temp;
    }
    printf("Sorted array is \n");
    for(i=0;i<n;i++)
    {
        printf("%d \t",a[i]);
    }
    getch();
}

```

3. Program to solve Tower of Hanoi using Recursion

```

#include<stdio.h>
#include<conio.h>
void hanoi(int N,char S,char I,char D);
int main()
{
    char S='a',I='b',D='c';
    int N;
    clrscr();
    printf("Enter the no of disk:");
    scanf("%d",&N);
    hanoi(N,S,I,D);
    getch();
    return 0;
}
void hanoi(int N,char S,char I,char D)
{
    if(N!=0)
    {
        hanoi(N-1,S,D,I);
        printf("\n move %d disk from %c to %c",N,S,D);
        hanoi(N-1,I,S,D);
    }
}

```

4. Program to search an element using recursive binary search technique

```

#include<stdio.h>
#include<conio.h>
int A[20];
int bsearch(int,int,int);
void main()
{
    int n,i,loc,ele,low,high;
    clrscr();
    printf("How many element");
    scanf("%d",&n);
    printf("Enter the element");
    for(i=0;i<n;i++)
    {
        scanf("%d",&A[i]);
    }
    printf("Enter the element that you want to search\n");
    scanf("%d",&ele);
    low=0;
    high=n-1;
    loc=bsearch(low,high,ele);
    if(loc>=0)

```

```

        {
            printf("Element found in position %d",loc+1);
        }
        else
        {
            printf("Search unsuccessful");
        }
        getch();
    }
    int bsearch(int low,int high,int ele)
    {
        int mid;
        mid=(low+high)/2;
        if(low>high)
        {
            return (-1);
        }
        else if(ele==A[mid])
        {
            return (mid);
        }
        else if(ele>A[mid])
        {
            return (bsearch(mid+1,high,ele));
        }
        else
        {
            return (bsearch(low,mid-1,ele));
        }
    }

```

5. Program to sort the given list using merge sort technique.

```

#include<stdio.h>
#include<conio.h>
void mergesort(int a[],int low,int high);
void merge(int a[],int low,int mid,int high);
void main()
{
    int a[40],i,n,j,low,high;
    clrscr();
    printf("Enter the size of th array");
    scanf("%d",&n);
    printf("Enter the element\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    low=0;
    high=n-1;
    mergesort(a,low,high);
    printf("After sorting \n");
    for(i=0;i<n;i++)
    {

```

```

        printf("%d\t",a[i]);
    }
    getch();
}
void mergesort(int a[],int low,int high)
{
    int mid;
    mid=(low+high)/2;
    if (low<high)
    {
        mergesort(a,low,mid);
        mergesort(a,mid+1,high);
        merge(a,low,mid,high);
    }
}
void merge(int a[],int low,int mid,int high)
{
    int c[20],i,j,k;
    i=low;

```

```

j=mid+1;
k=low;
while(i<=mid && j<=high)
{
    if(a[i]>a[j])
    {
        c[k]=a[j];
        k++;
        j++;
    }
    else
    {
        c[k]=a[i];
        i++;
        k++;
    }
}
while(i<=mid)
{
    c[k]=a[i];
    k++;
    i++;
}
while(j<=high)
{
    c[k]=a[j];
    k++;
    j++;
}
for(i=low; i<=high; i++)
{
    a[i]=c[i];
}
}

```

6. Program to sort the given list using quick sort technique

```

#include<stdio.h>
#include<conio.h>
void QUICKSORT(int x[],int low,int high);
int PARTITION(int x[],int low,int high);
void main()
{
    int a[10],n,i,low,high;
    clrscr();
    printf("Enter the size of the array");
    scanf("%d",&n);
    printf("Enter the element \n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    low=0;
    high=n-1;
    QUICKSORT(a,low,high);
    printf("Array after sorting \n");
    for(i=0;i<n;i++)
    {

```

```

        printf("%d\t",a[i]);
    }
    getch();
}

void QUICKSORT(int x[],int low,int high)
{
    int pos;
    if(low<high)
    {
        pos=PARTITION(x,low,high);
        QUICKSORT(x,low,pos-1);
        QUICKSORT(x,pos+1,high);
    }
}

int PARTITION(int x[],int low,int high)
{
    int key,temp,true=-1;
    int left,right;
    key=x[low];
    left=low+1;

```

```

right=high;
while(true)
{
    while((left<high) && (key>=x[left]))
    {
        left++;
    }
    while(key<x[right])
    {
        right--;
    }
    if(left<right)
    {
        temp=x[left];
        x[left]=x[right];
        x[right]=temp;
    }
    else
    {
        temp=x[low];

```

```

        x[low]=x[right];
        x[right]=temp;
        return(right);
    }
}
}

```

-
7. Program to implement Stack operations using arrays.


```

#include<stdio.h>
#include<conio.h>
#define MAXSIZE 5
int top=-1;
int stack[MAXSIZE];
void push();
void pop();
void display();
void main()
{
    int ch;
    clrscr();
    do
    {
        printf("STACK OPERATION\n");
        printf("_____N\n");
        printf("1.Push\n");
        printf("2.Pop\n");
        printf("3.Display\n");
        printf("4.Exit\n");
        printf("_____N\n");

```

```

        printf("Enter the choice\n");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
                break;
            default:
                printf("Invalid choice\n");
        }
    }while(ch>=1 && ch<=4);
    getch();

```

```

}
void push()
{
    int element;
    if (top==MAXSIZE-1)
    {
        printf("Stack full\n");
    }
    else
    {
        printf("Enter the element:\n");
        scanf("%d",&element);
        top++;
        stack[top]=element;
        printf("Element is pushed\n");
    }
}
void pop()
{
    int element;
    if (top== -1)

```

```

    {
        printf("Stack is empty\n");
    }
    else
    {
        element=stack[top];
        top--;
        printf("The popped element is:%d\n",element);
    }
}
void display()
{
    int i;
    if (top== -1)
    {
        printf("Stack is empty \n");
    }
    else
    {
        printf("Stack elements are \n");
        for(i=top; i>=0; i--)

```

```

        {
            printf("%d\n",stack[i]);
        }
    }
}

```

8. Program to evaluate given postfix expression.

```
#include<stdio.h>
#include<conio.h>
#include<ctype.h>
#include<math.h>
#define MAXSIZE 10
void push(float);
float pop();
float evalpost(char[],float[]);
float stack[MAXSIZE];
int top=-1;
void main()
{
    char postfix[20],ch;
    float value[20],result;
    int i=0;
    clrscr();
    printf("Enter a valid expression:");
    scanf("%s",postfix);
    while(postfix[i]!='\0')
    {
        ch=postfix[i];

        if(isalpha(ch))
        {
            printf("Enter the value of %c\n",postfix[i]);
            scanf("%f",&value[i]);
        }
        i++;
    }
    result=evalpost(postfix,value);
    printf("The result of calculation %s is=%f \n",postfix,result);
    getch();
}

void push(float val)
{
    top=top+1;
    stack[top]=val;
}

float pop()
{
    float val;
    val=stack[top];
    top=top-1;
}
```

```

    return(val);
}
float evalpost(char postfix[],float value[])
{
    char ch;
    int i=0;
    float op1,op2,res;
    while(postfix[i]!='\0')
    {
        ch=postfix[i];
        if(isalpha(ch))
        {
            push(value[i]);
        }
        else
        {
            op2=pop();
            op1=pop();
            switch(ch)
            {
                case '^':

```

```

                    push(pow(op1,op2));
                    break;
                case '*':
                    push(op1*op2);
                    break;
                case '/':
                    push(op1/op2);
                    break;
                case '+':
                    push(op1+op2);
                    break;
                case '-':
                    push(op1-op2);
                    break;
            }
        }
        i++;
    }
    res=pop();
    return(res);
}

```

9. Program to implement circular queue using array.

```

#include<stdio.h>
#include<conio.h>
#define MAX 5
int cqueue[MAX];
int front=-1,rear=-1;
void insert();
void display();
void delet();
void display();
void main()
{
    int ch;
    clrscr();
    do
    {
        printf("\n CQUEUE OPERATION \n");
        printf("_____ \n");
        printf("1.Insert\n");
        printf("2.Delet\n");
        printf("3.Display\n");
        printf("4.Exit\n");

        printf("_____ \n");
        printf("Enter youre choice:");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:
                insert();
                break;
            case 2:
                delet();
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
                break;
        }
    }while(ch>1 && ch<=4);
    getch();
}

```

```

}
void insert()
{
    int ele;
    if(front==(rear+1)%MAX)
    {
        printf("Circular queue is full! \n");
    }
    else
    {
        printf("Enter element \n");
        scanf("%d",&ele);
        rear=(rear+1)%MAX;
        cqueue[rear]=ele;
    }
    if(front==-1)
    {
        front=0;
    }
}
void delet()

```

```

{
    int ele;
    if(front==-1)
    {
        printf("Queue is empty");
    }
    else
    {
        ele=cqueue[front];
        printf("Deleted element is %d",ele);
        if(front==rear)
        {
            front=-1;
            rear=-1;
        }
        else
        {
            front=(front+1)%MAX;
        }
    }
}

```

```

void display()
{
    int i;
    if(front==-1)
    {
        printf("Queue is empty ");
    }
    else
    {
        if(front>rear)
        {
            for(i=front;i<=MAX-1;i++)
            {
                printf("%d\t",cqueue[i]);
            }
            for(i=0;i<rear;i++)
            {
                printf("%d\t",cqueue[i]);
            }
        }
        else

```

```

        {
            for(i=front;i<=rear;i++)
            {
                printf("%d\t",cqueue[i]);
            }
        }
    }
}

```

10. Program to implement Queue operations using arrays.

```

#include<stdio.h>
#include<conio.h>
#define MAX 5
int rear=-1,front=-1;
int Q[MAX];
void insert();
void display();
void delet();
void main()
{
    int ch;
    clrscr();
    do
    {
        printf("\n QUEUE OPERATION\n");
        printf("_____ \n");
        printf("1.Insert\n");
        printf("2.Delet\n");
        printf("3.Display\n");
        printf("4.Exit\n");
        printf("_____ \n");

```

```

        printf("Enter youre choice");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:
                insert();
                break;
            case 2:
                delet();
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
                break;
        }
    }while(ch>1 && ch<=4);
    getch();
}
void insert()

```



```

{
    int element;
    if(rear==MAX-1)
    {
        printf("Queue is full\n");
    }
    else
    {
        printf("Enter the element \n");
        scanf("%d",&element);
        rear=rear+1;
        Q[rear]=element;
        if(front==-1)
        {
            front=0;
        }
    }
}
void delet()
{
    int element;

```

```

    if(front==-1)
    {
        printf("Queue is empty\n");
    }
    else if(front==rear)
    {
        element=Q[front];
        printf("The deleted element is %d\n",element);
        front=-1;
        rear=-1;
    }
    else
    {
        element=Q[front];
        front=front+1;
        printf("The deleted element is %d\n",element);
    }
}
void display()
{
    int i;

```

```
    if (front== -1)
    {
        printf("Queue is empty\n");
    }
    else
    {
        printf("Element is the queue are \n");
        for(i=front; i<=rear; i++)
        {
            printf("%d\n", Q[i]);
        }
    }
}
```
